

Progress Report 4

Student Name: Advitiya Sharda

Student ID: 300395470

Project: Smart Door Anomaly Detection & Cybersecurity for Elderly Care

Course: CSIS 4495 – Applied Research Project, Section 3

Reporting Period: March 11 – March 16, 2026

Report Date: March 16, 2026

Work Date / Hours Logs

Date	Hours	Description of Work Done
Mar 11, 2026	2.0	Installed face_recognition (dlib backend). Ran diagnose_recognition.py to record baseline accuracy with OpenCV HOG (~73%), then re-ran after install to confirm dlib activation (~89%). Fixed interactive input() prompts in tests/test_integration.py and tests/test_facial_recognition.py — now auto-run and CI-safe.
Mar 12, 2026	2.0	Created calibrate_recognition.py to test 11 threshold values (0.30–0.80), select optimal via F1 score, and auto-write to config.py. Fixed confidence scoring bug in facial_recognition.py. Added fall-to-threat escalation in fall_detection_routes.py. Updated requirements.txt.
Mar 13, 2026	3.0	Added GE endpoint in aroutes.py . Implemented check_repeated_falls() in threat_detection.py . Added 10 unit tests in tests/test_facial_recognition.py.
Mar 15, 2026	3.0	Overhauled README.md: updated Python 3.11, Phase 2 instructions (3-terminal setup), fall detection + LSTM sections, updated routes and structure. Ran full integration tests and verified all components.
Mar 16, 2026	1.5	Report creation Team meeting Merge code into the main branch

Total Hours: 11.5

Summary Description of Work

This week I completed Phase 2 integration for the facial recognition and security threat systems.

I upgraded the recognition engine by enabling the dlib backend (face_recognition library), improving accuracy from ~73% to ~89% on our sample dataset. I also implemented a self-

calibrating threshold tool that determines optimal matching distance using real sample images and updates configuration automatically.

A key bug in confidence scoring was fixed, where the denominator was incorrectly hardcoded to 0.6. This caused inflated confidence values when using the OpenCV fallback. The fix ensures engine-specific threshold usage.

On the security side, I integrated fall detection into the threat system:

- Individual falls now escalate to **CRITICAL alerts**
- Added `check_repeated_falls()`:
 - 2 falls in 24h → **HIGH**
 - 3+ falls → **CRITICAL**

I also:

- Added a recognition status API endpoint
- Implemented 10 CI-safe unit tests (no webcam required)
- Completed a full Phase 2 README overhaul

As team lead, I managed the final merge of the respective branches into main and verified successful end-to-end integration.

Repo Check-in of Implementation

The following files were committed:

- `tests/test_integration.py` — auto-runs without input()
- `tests/test_facial_recognition.py` — 10 unit tests (no webcam required)
- `scripts/calibrate_recognition.py` — threshold calibration tool
- `api/facial_recognition.py` — confidence scoring bug fix
- `api/fall_detection_routes.py` — fall-to-threat escalation
- `api/routes.py` — new recognition status endpoint
- `api/threat_detection.py` — repeated fall detection logic
- `requirements.txt` — added `face_recognition>=1.3.0`
- `README.md` — full Phase 2 overhaul

All changes were committed with descriptive messages on branch week11_advitiya and merged into main via pull request.

AI Use Section

AI Tool	Version / Account	Feature Used	Value Added by Me
Cursor (AI Editor)	claude-4.6-sonnet (integrated)	Code generation (calibration script, tests, API endpoint)	Reviewed all code, verified logic, customized thresholds and error handling
ChatGPT	Pro Tier	Research (dlib vs OpenCV accuracy, fall risk patterns)	Adapted insights to HOG fallback architecture and elderly care context

Appendix: Prompt History

Prompt 1:

"What are the accuracy differences between dlib face encodings and OpenCV HOG for live webcam recognition?"

- Used: Understanding of 128-dim dlib encoding vs HOG
- Modified: Adjusted threshold tuning and calibration ranges for our dataset

Prompt 2:

"Clinical guidelines on repeated fall risk assessment for elderly residents in care facilities"

- Used: Fall frequency thresholds
- Modified: Converted into threat rules (2 → HIGH, 3+ → CRITICAL)

Prompt 3:

"How to write CI-safe Python unit tests that don't require hardware like a webcam"

- Used: Mocking hardware dependencies
- Modified: Applied to both dlib and OpenCV using synthetic NumPy arrays